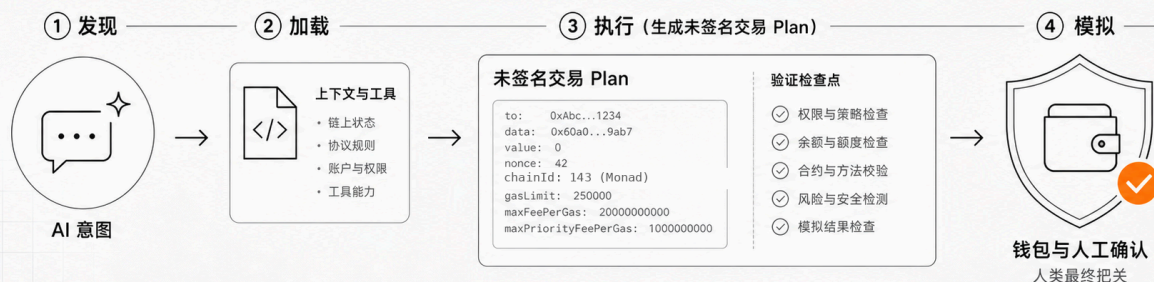


为什么 AI Agent 需要 Moss

从相信 AI 到签名前验证



为什么 AI Agent 需要 Moss：把链上执行从「相信 AI」变成「签名前验证」

Neo | Dev Builder (Tech) | 2026-07-14

QUOTE

AI Agent 可以理解「帮我把 1 MON 换成 USDC」，但理解一句话，不等于能安全地完成一笔链上交易。Moss 真正解决的，不是让 Agent 更快发交易，而是让每一笔交易在签名前都可构建、可解释、可模拟、可叫停。

—— Neo

本文看点

01

Moss 解决什么问题

02

Plan 如何被验证

03

为什么 Agent 需要它

如果你对 AI Agent 说：

「帮我在 Monad 上把 1 MON 换成 USDC。」

这句话对人很简单，对执行系统却一点也不简单。

Agent 必须知道应该调用哪个协议、合约地址是否正确、原生 MON 是否需要包装、代币有多少位小数、是否需要授权、授权给谁、最多授权多少、滑点如何限制，以及一组交易执行后究竟发生了哪些资产变化。

传统 Agent 很擅长生成文本，也可以调用工具，但链上交易容不得「大概正确」。一个错误地址、一次无限授权、一处精度换算错误，或者一笔没有被发现的额外资金流，都可能造成不可逆损失。

这正是 Moss 想解决的问题。

01 DEFINITION

一、Moss 是什么？

Moss 是一个面向 Monad 的 AI Agent 链上协议能力层。它把不同 DApp 和协议的复杂交互，抽象成 Agent 可以统一调用的能力：



```
text
```

```
discover → load → action → simulate
```

它让 Agent 不再直接拼装 ABI、合约地址、calldata、multicall 和代币精度，而是用人类可读的参数表达意图，由协议适配器构建未签名交易，再由模拟器验证实际效果。

Moss 的职责边界非常明确：

- 构建交易，但不签名；
- 模拟交易，但不发送；
- 提取真实 effects，但不替用户做最终决定；
- 发现 warning，就在签名前停止。

因此，Moss 不是钱包，也不是自动交易机器人。更准确地说，它是 Agent 与钱包签名器之间的「协议能力层 + 签名前验证层」。

截至本文写作时，Moss 仍处于 Alpha 阶段，目标网络是 Monad 主网，已支持 WMON、通用 ERC-20、通用 ERC-721 和 Kuru 等能力。它未经安全审计，API、Plan 格式和包结构仍可能变化，不应把它描述成已经成熟的生产基础设施。

02 PROBLEM

二、它解决的不是「不会调用」，而是「不能验证」

很多 Agent 框架已经能调用合约。问题在于：调用成功，只能证明交易没有回滚，不能证明它符合用户原始意图。

假设一个 Swap 工具返回两笔交易：

- 1 授权 USDC；
- 2 调用 DEX Router。

仅检查交易是否能执行，回答不了这些问题：

- 授权额度是不是刚好够用，还是无限授权？
- spender 是目标协议，还是陌生地址？
- 用户最多同意付出多少资产？
- 用户至少应该收到多少资产？
- 是否发生了 Plan 没有声明的额外转账？
- 交易做了它自己声称要做的事，但这件事真的是用户要求的吗？

Moss 的核心设计，是把「我打算做什么」和「执行后实际发生什么」分开，再进行机械对账。

这比单纯依赖 Prompt 更可靠。Prompt 可以告诉 Agent 「小心一点」，但无法像程序一样，对每一笔流出、流入和授权做确定性比较。

三、四步工作流：从意图到可验证 Plan

1. discover：先找能力，不猜合约

Agent 先按用户视角的动作搜索能力，例如 `swap`、`wrap`、`transfer`、`supply`。

Moss 特意把用户语义与合约函数名分开。WMON 合约中的函数名可能是 `deposit()`，但用户的意图是 `wrap`。这种统一词汇让 Agent 面向「资金动作」思考，而不是被不同协议的函数命名绑架。

2. load：读取调用契约

找到候选能力后，Agent 通过 `load` 获取：

- intent 模板；
- 参数语义；
- 风险标签；
- 调用所需的人类可读输入。

例如金额可以传入 `"1.5"`，运行时负责按代币 decimals 转成链上整数。Agent 不需要自己计算 wei，也不应该凭记忆从网络搜索代币地址。

Moss 对代币符号还有一条重要安全规则：`USDC` 这样的符号只通过维护过的 TokenTable 解析，不会退回链上调用 `symbol()` 猜测地址。因为任何人都可以部署一个名字同样叫 USDC 的假币。未知符号会直接报错，而不是「智能猜测」。

3. action：生成未签名 Plan

`action` 不发送交易，而是返回一个 Plan。简化后可以理解为：

```
json
{
  "intent": "Swap 1 MON into USDC",
  "txs": ["unsigned transaction steps"],
  "expects": {
```

```
"out": [{ "token": "MON", "amountMax": "1" }],
"in": [{ "token": "USDC", "amountMin": "quoted amount" }],
"approvals": []
},
"planHash": "0x..."
}
```

这里最重要的不是 `txs`，而是 `expects`。

风险标签只能说「这笔交易涉及资金流出」或「需要授权」，却不能回答流出多少、授权给谁。
`expects` 把风险变成机器可比较的边界：最多付出多少、至少收到多少、允许给哪个 spender 授权，以及授权上限是多少。

`planHash` 则覆盖 `chainId`、`account`、交易步骤、`expects` 和 `confirms` 等关键内容。如果 Plan 在 Agent 与模拟器之间被改动，重新计算的哈希不一致，就会触发 `PLAN_TAMPERED`。

4. simulate : 把 declared 变成 verified

每个 Plan 都必须经过 `simulate`。这是 Moss 的验证之门，而不是可选的预览按钮。

Moss 没有只做一次普通 `eth_call`。根据项目的 ADR，它在 Monad 上使用 `debug_traceCall` 的双 tracer 方案：

- `callTracer` 配合日志，获得调用树、事件和原生 MON 的 value 流；
- `prestateTracer` 的 diff mode 获取状态前后差异；
- 将状态差异累积到 `stateOverrides`，使后一笔交易能在前一笔的模拟结果上继续执行。

因此，`simulate([planA, planB])` 可以验证 `claim → swap → supply` 这样的多步流程。Plan B 能使用 Plan A 在模拟状态里刚得到的资产，而每个 Plan 的 `expects` 仍然独立对账。

模拟器会从 trace 中提取：

- ERC-20 与 ERC-721 转移；
- 代币授权和 NFT operator 授权；
- 原生 MON 的流入流出；
- wrapped token 的 mint / burn；
- 收款方、回滚和协议回执。

然后将实际 effects 与 Plan 的 expects 比较。未声明流出、超额流出、未声明授权、超额授权、最低流入未满足、NFT 异常移动、Plan 被篡改或交易回滚，都会变成 warning。

核心规则只有一句：

「Any undeclared difference becomes a warning. 任何未声明的差异都必须被看见。」

04 WHY MOSS

四、为什么 AI Agent 尤其需要 Moss ?

第一，LLM 是概率系统，资产执行需要确定性边界

大模型的优势是理解模糊语言、拆解目标和选择工具，但它的输出具有概率性。链上资产执行则要求地址、金额、授权和调用顺序精确无误。

正确的系统设计，不是要求 LLM 永远不犯错，而是默认它可能犯错，并用确定性程序把错误拦在签名前。

Moss 的分工是：Agent 负责理解与协调，Adapter 负责正确构建，Simulator 负责机械验证，Wallet 负责签名，用户保留最终决定权。

第二，工具调用成功不等于用户意图实现

Moss 明确区分两种检查：

- 1 **Effects reconciliation**：服务器侧把 Plan 声明与模拟 effects 机械对账；
- 2 **Intent alignment**：Agent 把 effects 与用户原话比较。

这一区分非常关键。

零 warning 只说明「Plan 做了它声明要做的事」，不说明「Plan 做的是用户要求的事」。如果用户说 Swap，Agent 却选择了 Supply，即使交易的资金流完全符合这个 Supply Plan，也必须停止。

机械验证看不到用户最初说了什么，只有 Agent 持有原始意图。因此，Moss 没有试图用一层检查解决所有问题，而是把可机械验证的部分与必须语义判断的部分拆开。

第三，签名权必须与推理权分离

Moss 永不持有私钥，永不签名，永不发送交易。它输出的是未签名 Plan，最终交易必须由外部钱包重新检查账户、网络和余额，并由用户确认。

这意味着，即使 Agent、适配器或模拟流程出现问题，系统仍保留一个独立的最终信任边界。

对于真实资产，Agent 最合理的权限不是「替用户做决定并签名」，而是「生成受约束、可解释、可验证的提案」。

05

BOUNDARIES

五、一个容易被忽视的细节：模拟通过不等于执行结果承诺

Moss 的模拟使用当下的链上状态。模拟完成到用户签名之间，价格、订单簿、流动性和合约状态都可能变化。

所以，零 warning 不是「这笔交易一定成功」，更不是收益保证。它只表示在模拟时点：

- Plan 未被篡改；
- 实际 effects 没有越过声明边界；
- 没有发现未声明资金流或授权；
- 必要的协议回执出现了。

真正执行时仍需要合约内的 `minAmountOut` 等保护，钱包也要重新检查余额与交易内容。模拟是一张安全网，不是未来状态的预言机。

Moss v1 还明确不支持 Permit / EIP-2612 签名流、跨链桥和单笔交易内的闪电贷原子组合。尤其是跨链桥，目标链效果无法通过源链模拟完整验证，这与 Moss 要把 declared 变成 verified 的原则冲突。

这些限制不是缺点清单里的小字，而是安全模型的一部分。一个可信的开源项目，不只应该告诉你它能做什么，也应该明确告诉你它拒绝做什么。

06 PRACTICE

六、从读代码到做贡献：我如何理解 Moss

我最初是从 README 的四步流程进入项目，然后继续阅读 `packages/core`、`packages/simulator`、`packages/erc`、协议模板和 WMON 参考适配器。

真正让我理解 Moss 的，不是背下四个工具名，而是看到它如何把安全边界编码进类型、Plan 和测试：

- `@Capability` 声明用户视角的意图、参数和风险；
- `plan()` 必须给出量化 expects；
- TokenTable 拒绝同符号不同地址的静默覆盖；
- `observations` 只负责用协议语言叙述结果，不能覆盖审计 warning；
- 任意 warning 都触发 halt rule，不能通过反复模拟把它当噪声消掉。

为了验证自己是否真的理解，我还基于协议模板做了一个 MockVault Adapter 草稿，尝试描述 Vault 的 `deposit / withdraw`、Plan 和 expects。这个练习很快暴露出：做一个 Demo，与设计通用 ERC-4626 Interface Layer 完全不是同一难度。

通用接口还要回答 Vault 如何实例化、`deposit / mint / withdraw / redeem` 如何映射用户语义、asset 与 share 的 expects 如何量化，以及 ABI 来源怎样保持可追溯。

因此，我把自己的首个贡献计划收敛到 Issue #13 的 ERC-4626 基础层：先与 Maintainer 对齐范围，再从 `IERC4626.sol` → `ERC4626Abi` → `tests / docs` → Draft PR 做一个小而可审查的改动，而不是未经讨论就提交一个巨大的通用 Adapter。

这也是 Moss 给我的另一个启发：安全不仅存在于运行时，也存在于开源协作方式中。ADR、ABI provenance、量化 expects、live e2e、changeset 和小步 PR，本质上都在减少「看起来差不多」进入主分支的机会。

七、Moss 适合哪些场景？

如果继续发展，Moss 可以成为多类 Agent 应用的基础执行层：

- DeFi 助手：在 Swap、Wrap、Transfer 前构建并验证交易；
- 自动化金库：在再平衡、领取收益或调整仓位前核对资产流；
- DAO 财库：为提案执行生成可审计、可模拟的 Plan；
- 多步骤策略：验证 `claim → swap → supply` 等跨协议流程；
- Agent Wallet：在 Agent 推理层与钱包签名层之间增加可验证边界；
- Monad 协议生态：通过标准 Adapter 暴露 Agent 可发现的能力。

但现阶段更适合把 Moss 当成一个值得研究、测试和参与贡献的 Alpha 开源项目，而不是直接托管大额真实资产。



结语：不要让 Agent 更像钱包，要让它更像受约束的提案者

AI Agent 进入 Web3 后，真正困难的问题不是「它能不能调用合约」，而是：当它开始接近真实资产，我们如何知道它将要做什么、实际会发生什么，以及发现偏差时能否及时停下。

Moss 给出的答案不是让模型变得绝对可靠，而是重新设计职责：

「Agent 理解意图，协议适配器构建 Plan，模拟器核对 effects，钱包保管私钥，用户做最终决定。」

从这个角度看，Moss 的核心价值并不是自动化交易，而是把链上执行从「相信 AI」推进到「签名前验证」。

这可能才是 AI Agent 真正进入开放金融系统之前，需要补上的基础设施。

参考资料

- Moss GitHub : <https://github.com/nishuzumi/moss>
- 中文 README : <https://github.com/nishuzumi/moss/blob/main/README.zh-CN.md>
- Getting Started : <https://github.com/nishuzumi/moss/blob/main/docs/getting-started.md>
- Agent Skill Guide : <https://github.com/nishuzumi/moss/blob/main/docs/agent-skill.md>
- Security Model : <https://github.com/nishuzumi/moss/blob/main/SECURITY.md>
- ADR 0002 — Simulation via `debug_traceCall` :
<https://github.com/nishuzumi/moss/blob/main/docs/adr/0002-simulation-via-debug-tracecall.md>
- ADR 0004 — Quantified Expects :
<https://github.com/nishuzumi/moss/blob/main/docs/adr/0004-quantified-expects-in-plans.md>
- ADR 0005 — Curated Token Catalog :
<https://github.com/nishuzumi/moss/blob/main/docs/adr/0005-curated-token-catalog.md>

作者：Neo | Dev Builder (Tech)

E N D

我是 Neo，一名关注 AI Agent 安全执行与 Monad 生态的 Dev Builder。

如果你觉得今天这篇有收获，欢迎**点赞、在看、转发**三连，我们下篇见。